P1633R0
Amendments to the C++20 Synchronization Library

Published Proposal, 2019-06-15

Authors:

David Olsen (NVIDIA)

Olivier Giroux (NVIDIA)

Source:

GitHub

Issue Tracking:

GitHub

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

SG1

Table of Contents

§ 1. Introduction

During the wording review of the C++20 Synchronization Library, [P1135R4], four design flaws were found in the paper. Rather than send the entire paper back to SG1 to look over the changes and risk missing the deadline for C++20, this new paper is being written for SG1 to review.

The wording changes here have already been applied to [P1135R5]. If SG1 approves these changes, then P1135 will go to LWG in its current state. If any of the changes are rejected by SG1, then the change will be backed out of P1135, by applying the wording change in this paper in reverse, before LWG gives its final approval to P1135.

§ 2. Changelog

Revision 0: Initial version.

§ 3. Make atomic_flag::test const

§ 3.1. Motivation

`atomic_flag::test` does not modify the `atomic_flag` object at all, so it should be a `const` member function. Similarly, the first parameter to `atomic_flag_test` and `atomic_flag_test_explicit` should be of type `const atomic_flag*` or `const volatile atomic_flag*`.

This bug seems to have been here from the beginning. See [P0995R0]. There is no record of a discussion of the const-ness of these functions.

§ 3.2. Wording

Modify the header synopsis for `<atomic>` in **[atomics.syn]** as follows:

```
  // 30.9, flag type and operations
  struct atomic_flag;
  bool atomic_flag_test(const volatile atomic_flag*) noexcept;
  bool atomic_flag_test(const atomic_flag*) noexcept;
  bool atomic_flag_test_explicit(const volatile atomic_flag*, memory_orde
  bool atomic_flag_test_explicit(const atomic_flag*, memory_order) noexcep
```

Modify **[atomics.flag]** as follows:

30.9 Flag type and operations **[atomics.flag]**

```
namespace std {
  struct atomic_flag {
    bool test(memory_order = memory_order::seq_cst) const volatile noexcep
    bool test(memory_order = memory_order::seq_cst) const noexcept;

    // ...
  };

  bool atomic_flag_test(const volatile atomic_flag*) noexcept;
  bool atomic_flag_test(const atomic_flag*) noexcept;
  bool atomic_flag_test_explicit(const volatile atomic_flag*, memory_orde
  bool atomic_flag_test_explicit(const atomic_flag*, memory_order) noexcep

  //...
}
```

Still within section **[atomics.flag]**, change the function signatures between paragraph 4 and paragraph 5 as follows:

```
bool atomic_flag_test(const volatile atomic_flag* object) noexcept;
bool atomic_flag_test(const atomic_flag* object) noexcept;
bool atomic_flag_test_explicit(const volatile atomic_flag* object, memory_
bool atomic_flag_test_explicit(const atomic_flag* object, memory_order orc
bool atomic_flag::test(memory_order order = memory_order::seq_cst) const v
bool atomic_flag::test(memory_order order = memory_order::seq_cst) const r
```

§ 4. Prohibit counting_semaphore<-1>

§ 4.1. Motivation

```
template<ptrdiff_t least_max_value = implementation-defined>
  class counting_semaphore;
```

Template class `counting_semaphore` has a non-type template parameter `least_max_value` which is intended to put an upper limit on the number of times a semaphore of that type can be simultaneously acquired.

[P1135R3] had no restrictions on the value of `least_max_value`. There was nothing that prevented users from using `counting_semaphore<0>` or `counting_semaphore<-20>`, neither of which can do anything useful.

§ 4.2. Wording

Insert a new paragraph after paragraph 1 in [thread.semaphore.counting.class]:

> `least_max_value` shall be greater than zero; otherwise the program is ill-formed.

§ 5. Prohibit barrier::arrive(0)

§ 5.1. Motivation

[P0666R2] and early versions of P1135 did not put any lower limit on the value of the `update` parameter for `barrier::arrive(ptrdiff_t update)`. While working on [P1135R4], wording was added to require that `update >= 0`, since negative values don’t make sense. During LWG wording review in Kona, Dan Sunderland pointed out that `barrier::arrive(0)` would be problematic for implementations that used a fan-in strategy rather than a counter, since it would allow threads to wait on the barrier without arriving at the barrier. `arrive(0)` is of dubious usefulness even without the implementation problem, so the lower bound of `update` is changed from zero to one, making `arrive(0)` undefined behavior, the same as `arrive(-1)`.

§ 5.2. Wording

Change paragraph 13 in [thread.coord.barrier.class] as follows:

> `[[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);`
>
> *Expects:* `update` ~~`>=`~~`>` `0` is `true`, and `update` is less than or equal to the expected count for the current barrier phase.

§ 6. Allow latch::try_wait() to fail spuriously

§ 6.1. Motivation

The old wording for `latch::try_wait` of "*Returns:* `counter == 0`" implied that implementations needed to use `memory_order::seq_cst` for `counter` so that `try_wait` would immediately see the result of a different thread’s call to `count_down`. The new wording that allows `try_wait` to spuriously return `false` frees the implementation to use a more relaxed memory order.

§ 6.2. Wording

Change paragraph 13 in [thread.coord.latch.class] as follows:

> `bool try_wait() const noexcept;`
>
> *Returns:* With very low probability `false`. Otherwise `counter == 0`

§ References

§ Informative References

[P0666R2]
Olivier Giroux. Revised Latches and Barriers for C++20. 6 May 2018. URL: https://wg21.link/p0666r2

[P0995R0]
JF Bastien, Olivier Giroux, Andrew Hunter. Improving atomic_flag. 17 March 2018. URL: https://wg21.link/p0995r0

[P1135R3]
Bryce Adelstein Lelbach, Olivier Giroux, JF Bastien, Detlef Vollmann. The C++20 Synchronization Library. 21 January 2019. URL: https://wg21.link/p1135r3

[P1135R4]
Bryce Adelstein Lelbach, Olivier Giroux, JF Bastien, Detlef Vollmann, David Olsen. The C++20 Synchronization Library. 4 March 2019. URL: https://wg21.link/p1135r4